

› Lab - 1 | 22 January 2026

↳ 6 cells hidden

› Lab - 2 | 29 January 2026

↳ 14 cells hidden

› LAB -3 5 Feb

↳ 2 cells hidden

✓ Lab- 4 | 12 Feb

Binary Search

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

import seaborn as sns
import matplotlib.pyplot as plt
```

```
file_path = "/spam_or_not_spam.xlsx"
df = pd.read_excel(file_path)

print(df.head())
print(df.info())
```

		email	label
0	date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...		0.0
1	martin a posted tassos papadopoulos the greek ...		0.0
2	man threatens explosion in moscow thursday aug...		0.0
3	klez the virus that won t die already the most...		0.0

```
4   in adding cream to spaghetti carbonara which ...    0.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3016 entries, 0 to 3015
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ---
 0   email   3015 non-null    object
 1   label   3000 non-null    float64
dtypes: float64(1), object(1)
memory usage: 47.3+ KB
None
```

```
df = df.dropna(subset=["email", "label"])

X = df["email"]
y = df["label"]

print("Dataset shape:", df.shape)
```

```
Dataset shape: (2999, 2)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1,2)
)

X_train = vectorizer.fit_transform(X_train).toarray()
X_test = vectorizer.transform(X_test).toarray()

print("X_train shape:", X_train.shape)
```

```
X_train shape: (2399, 5000)
```

```
y_train = np.array(y_train).reshape(-1,1)
y_test = np.array(y_test).reshape(-1,1)
```

```
input_size = X_train.shape[1]
```

```
np.random.seed(42)

W = np.random.randn(input_size, 1) * 0.01
b = np.zeros((1,1))

learning_rate = 0.1
epochs = 500
```

```
def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))
```

```
def forward(X):
    Z = np.dot(X, W) + b
    A = sigmoid(Z)
    return A
```

```
def compute_loss(A, y):
    m = y.shape[0]
    epsilon = 1e-8

    loss = - (1/m) * np.sum(
        y*np.log(A+epsilon) +
        (1-y)*np.log(1-A+epsilon)
    )
    return loss
```

```
def backward(X, y, A):
    m = X.shape[0]

    dZ = A - y
    dW = (1/m) * np.dot(X.T, dZ)
    db = (1/m) * np.sum(dZ)

    return dW, db
```

```
loss_history = []
accuracy_history = []

for i in range(epochs):

    # Forward
```

```
A = forward(X_train)

# Loss
loss = compute_loss(A, y_train)
loss_history.append(loss)

# Accuracy on training set
predictions = (A >= 0.5).astype(int)
train_acc = np.mean(predictions == y_train)
accuracy_history.append(train_acc)

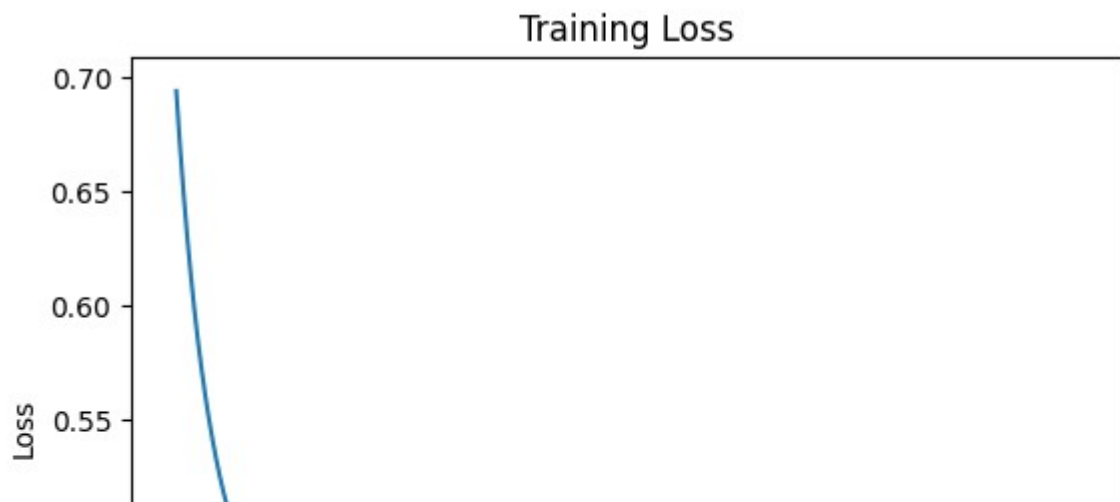
# Backward
dW, db = backward(X_train, y_train, A)

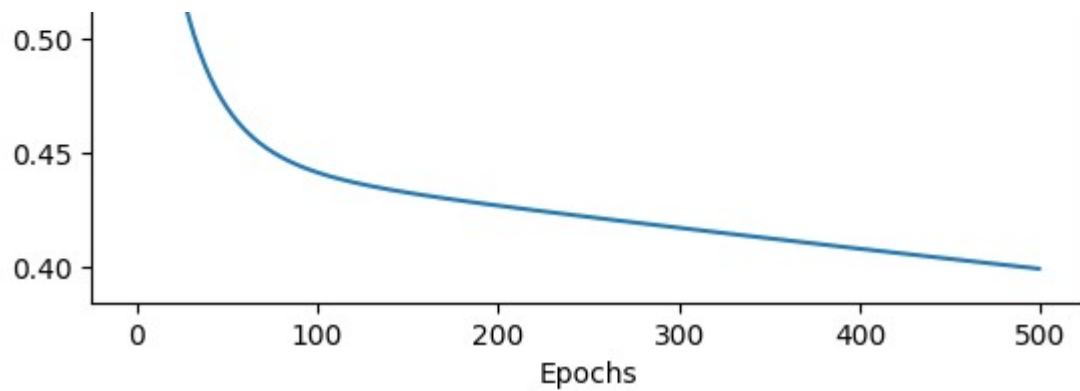
# Update
W -= learning_rate * dW
b -= learning_rate * db

if i % 100 == 0:
    print(f"Epoch {i}, Loss: {loss}, Accuracy: {train_acc}")
```

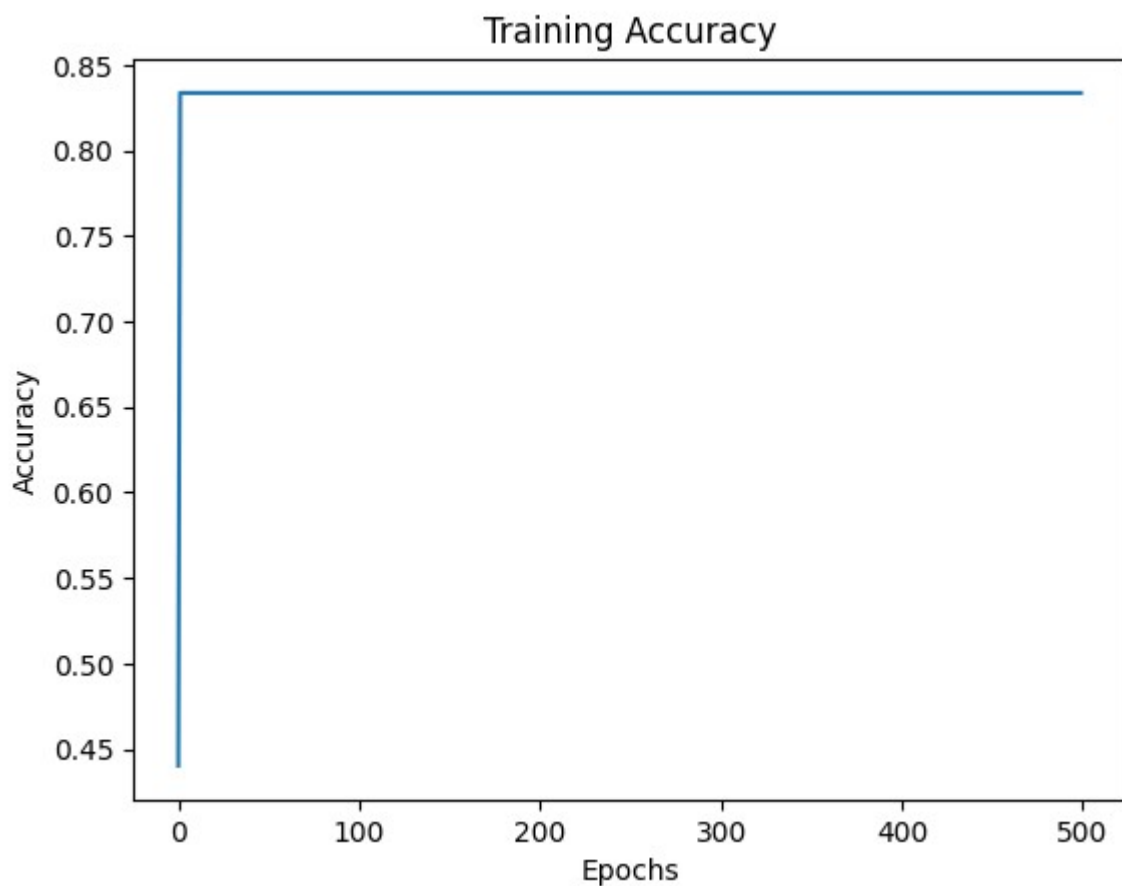
```
Epoch 0, Loss: 0.6938855908528171, Accuracy: 0.44018340975406417
Epoch 100, Loss: 0.441471697541632, Accuracy: 0.8336807002917882
Epoch 200, Loss: 0.426923304626026, Accuracy: 0.8336807002917882
Epoch 300, Loss: 0.417202053348458, Accuracy: 0.8336807002917882
Epoch 400, Loss: 0.4080324896469931, Accuracy: 0.8336807002917882
```

```
plt.figure()
plt.plot(loss_history)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
```





```
plt.figure()
plt.plot(accuracy_history)
plt.title("Training Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()
```



```
def predict(X, threshold=0.3):
    A = forward(X)
    return (A >= threshold).astype(int)
```

```
y_pred = predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 0.835
Precision: 1.0
Recall: 0.01
F1 Score: 0.019801980198019802
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.83	1.00	0.91	500
1.0	1.00	0.01	0.02	100
accuracy			0.83	600
macro avg	0.92	0.51	0.46	600
weighted avg	0.86	0.83	0.76	600

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Confusion Matrix

